

Chapter 6

Software Quality Management

Contents:

Quality Concerns	2
Total Quality Management (TQM):	2
Six Sigma:.....	7
Software Quality:	18
The Place of Software Quality in Project/Software Planning “step wise framework”:	24
ISO 9126:.....	26

Quality Concerns

- We need to define what qualities we require of a system.
- We also need to judge whether a system meets our quality requirements (this needs measurements).
- We may want to assess the likely quality of the final system while it is still under development.
- We also will need to make sure that the development methods would produce the quality needed.
- A potential customer might check that the suppliers were using the best development methods.

Total Quality Management (TQM):

Total Quality Management (TQM) is a management approach that emphasizes continual improvement throughout all aspects of an organization's operations, including software development.

TQM principles can be applied to software quality management to improve the quality of software products and services.

Key principles of TQM in software quality management:

- **Customer focus:** The focus of TQM is on meeting the needs and expectations of customers.

This means that software development teams should involve customers in the development process to ensure that the software meets their needs.

- **Continuous improvement:** TQM is a continuous process of improvement.

This means that software development teams should constantly be looking for ways to improve their processes, products, and services.

- **Employee involvement:** TQM emphasizes the involvement of all employees in the improvement process.

This means that all employees in the software development organization, from the engineers to the managers, should be involved in identifying and implementing improvements.

- **Process orientation:** TQM focuses on improving processes rather than simply inspecting products for defects.

This means that software development teams should **focus on identifying and eliminating defects** in the development process rather than simply relying on testing to identify and fix defects.

- **Data-driven decision making:** TQM emphasizes the **use of data to make decisions**.

This means that software development teams should **collect and analyze data to identify areas for improvement** and to measure the effectiveness of their improvements.

Benefits of TQM in software quality management:

- **Improved customer satisfaction:** TQM can help to improve customer satisfaction **by ensuring that software products and services meet the needs and expectations of customers**.
- **Reduced costs:** TQM can help to **reduce costs by preventing defects from occurring in the first place**.

- **Improved productivity:** TQM can help to improve productivity by streamlining processes and eliminating waste.
- **Increased innovation:** TQM can help to increase innovation by encouraging employees to identify and implement new ideas.

TQM tools and techniques:

- **Process mapping:** Process mapping is a tool that can be used to identify and understand the steps in a software development process.
- **Root cause analysis:** Root cause analysis is a technique that can be used to identify the root cause of a problem.
- **Statistical process control:** Statistical process control is a technique that can be used to monitor and control the quality of a process.
- **Benchmarking:** Benchmarking is a process of comparing the performance of one organization to the performance of other organizations.

Examples of TQM in software quality management:

- A software development team uses process mapping to identify a bottleneck in the development process. The team then implements a new process that reduces the time it takes to develop software.
- A software development team uses root cause analysis to identify the cause of a recurring software defect. The team then fixes the root cause of the defect, which prevents it from recurring.
- A software development team uses statistical process control to monitor the quality of its software releases. The team then uses the data from the statistical process control charts to identify and implement improvements to the development process.
- A software development team benchmarks its performance against the performance of other software development teams. The team then uses the benchmarking data to identify areas for improvement.

Six Sigma:

Six Sigma is the process of producing high and improved quality output.

This can be done in two phases – identification and elimination.

The cause of defects is identified and appropriate elimination is done which reduces variation in whole processes.

A six sigma method is one in which 99.99966% of all the products to be produced have the same features and are of free from defects.

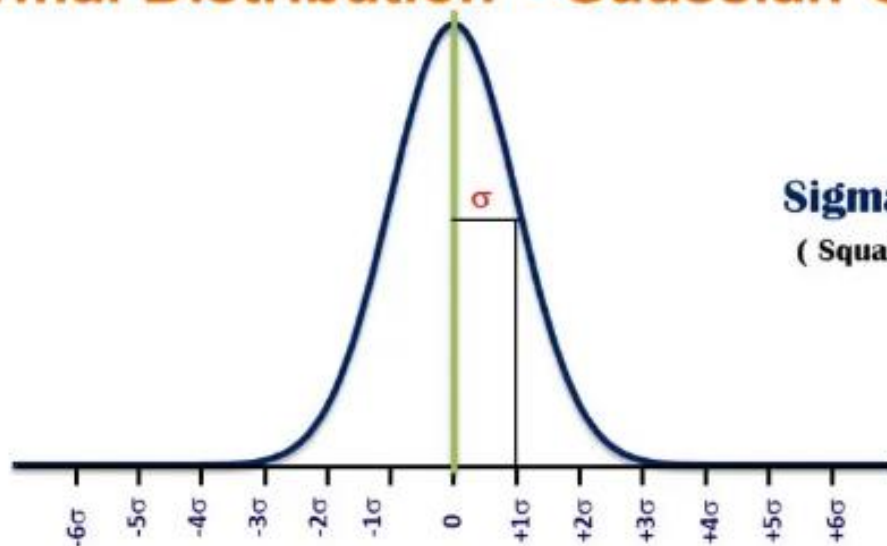
- σ – is a Greek symbol that indicated Standard Deviation, a measure of variability.
- Variability
 - No two things are alike and Everything is different.
 - The product we produce change continuously.
 - Variation in a product and process can be always measured.

- **Six Sigma** – A quality improvement strategy that focuses on eliminating defects through reduction of process variations.

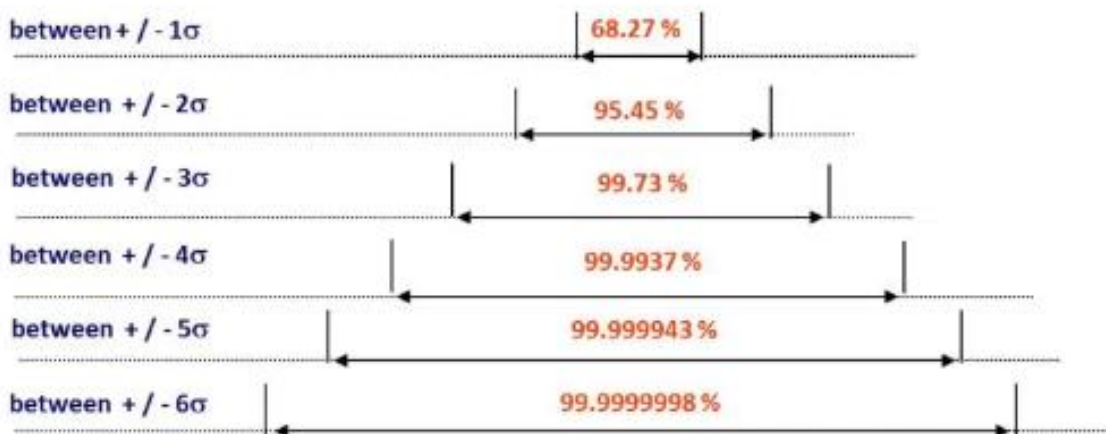
A product is said to be of **6sigma (6σ)** quality **if there are no more than 3.4 defects per million opportunities(DPMO)** at the part and process step level.

Six Sigma

Normal Distribution - Gaussian Curve



Sigma = σ = Deviation
(Square root of variance)



History:

The methodology is founded to eliminate product defects in manufacturing processes.

In the **mid-1980s**, **Motorola engineers** thought that the traditional quality assessment techniques weren't accurate enough since they measured defects in thousands of products.

Thus, they developed a new method called "Six Sigma" and started to measure defects in million products.

After the implementation of Six Sigma, Motorola has documented more than \$16 billion in savings in twelve years.

Other large companies like **Texas Instruments and General Electric** have also started to adopt this methodology.

In the **1990s**, **General Electric** introduced this approach in management processes to reduce deviations in processes.

Six Sigma Methodology

The six sigma uses two methodologies for completing projects or problem solving.

1.DMAIC Methodology

It specifies a data-driven quality strategy for improving processes.

This methodology is **used for existing process/system improvement**.

DMAIC stands for – Define, Measure, Analyze, Improve, and Control.

The DMAIC project methodology has five phases:

1. Define:

- In this stage, the project team **defines the problem or opportunity for improvement**.
- It involves clearly outlining the project goals, objectives, scope, and the customer requirements.
- Defining the problem is crucial for focusing efforts on the most critical issues and ensuring that everyone involved has a clear understanding of the project's purpose.

2. Measure:

- The Measure stage involves quantifying and gathering data related to the process in question.
- This step helps establish a baseline for the current performance of the process.
- Measurements might include key process metrics and indicators.
- Data collected during this stage serves as a basis for analysis in later stages.

3. Analyze:

- In the Analyze stage, the project team uses statistical tools and methods to analyze the data collected in the Measure stage.
- The goal is to **identify patterns, root causes of defects, and opportunities for improvement.**
- This step helps in understanding the relationships between variables and determining the factors that contribute to process variation.

4. Improve:

- The Improve stage focuses on **implementing changes to address the root causes** identified in the Analyze stage.
- The team develops and tests potential solutions, aiming to optimize the process and achieve the desired improvements.
- The emphasis is on making data-driven decisions to ensure that the implemented changes are effective.

5. Control:

- The Control stage is about establishing controls to sustain the improvements made during the previous stages.
- It involves developing a monitoring plan to track key process metrics and implementing mechanisms to prevent the recurrence of the identified issues.
- The goal is to ensure that the improved process remains stable and continues to meet customer requirements over the long term.

2.DMADV Methodology

It is used for new process/system development from scratch.

DMADV stands for – Define, Measure, Analyze, Design, and Verify.

The DMADV project methodology has five phases:

1. Define:

- Similar to DMAIC, the Define phase in DMADV involves clearly defining the goals and objectives of the project, but it **focuses on new processes or products rather than improvements to existing ones.**

2.Measure:

- Measure and quantify customer needs and specifications.
- This phase is critical for understanding the requirements that the new process or system must meet.

3. Analyze:

- Analyze the data gathered in the Measure phase to identify critical characteristics and features that the new process or system must possess.

4. Design:

- Develop and design the new process or system based on the analysis of customer needs and specifications.
- This phase is about creating a detailed plan for implementation.

5. Verify:

- Validate and verify that the designed process or system meets the defined requirements.
- This involves testing and confirming that the new process or system performs as intended.

Six Sigma Belt Levels | Six Sigma Certification

The Six Sigma certification levels are often compared to martial arts belt levels, indicating the level of expertise and responsibilities of individuals within the Six Sigma framework.

Six Sigma belt level:

1. White Belt:

- Entry-level certification.
- Provides basic knowledge of Six Sigma concepts or methodology.
- White Belts work with local quality problem teams but are not typically part of Six Sigma project teams.

2. Yellow Belt:

- Participants within the Six Sigma project as team members.
- Review project-supporting process improvements.
- Collaborate on projects led by Green Belts or Black Belts.

3. Green Belt:

- Leads Green Belt project teams.
- Assists in collecting and analyzing data for Black Belt projects.
- Generally, works on process improvement projects and supports Black Belts.

4. Black Belt:

- Leads projects for quality problem-solving.
- Trains and educates Six Sigma project teams.
- Analyzes and solves complex problems within the organization.
- Typically responsible for driving significant improvements in processes.

5. Master Black Belt:

- Trains and coaches Green and Black Belts.
- Serves as a Six Sigma expert and consultant for the organization.
- Often involved in shaping the overall Six Sigma strategy at the organizational level.
- Has a deep understanding of the methodology and plays a leadership role in deploying Six Sigma across various projects.

Six Sigma Benefits

It helps employers businesses and employees in many ways that are listed below:

- Reduced process variation.
- Reduced process waste (non-value added activities).
- Increased process efficiency.
- Increased revenue, and lowering cost of capital.
- Reduced quality defects and errors/mistakes.
- Increased brand value.
- Enhanced customer satisfaction.
- Leadership opportunities.
- Higher pay and job security.
- Increase in opportunities for career growth.

Software Quality:

Software quality refers to the **degree** to which a **software product or system meets specified requirements and customer expectations.**

It encompasses various attributes such as **functionality, reliability, usability, efficiency, maintainability, and portability.**

Achieving high software quality is crucial to ensure that the software meets its intended purpose, performs reliably, and can be maintained and updated effectively.

Key aspects that conclude software quality include:

1. Functionality:

The software **should fulfill its intended purpose and meet the specified requirements.**

It should provide the features and capabilities that users expect.

2. Reliability:

The software **should perform consistently and reliably under various conditions.**

It should **not crash, hang, or produce incorrect results.**

3. Usability:

The software should be **user-friendly and easy to use**.

It should have an intuitive interface, clear documentation, and provide a positive user experience.

4. Efficiency:

The software should **execute tasks in a timely and resource-efficient manner**.

It should **not consume excessive system resources, such as CPU, memory, or disk space**.

5. Maintainability:

The software should be **easy to maintain and update**.

This **includes** having well-organized code, clear documentation, and modular design, which facilitates changes and enhancements.

6. Portability:

The software should be **able to run on different platforms and environments without requiring major modifications**.

7. Scalability:

The software **should be able to handle increased load** or user demands by scaling its performance or capacity.

To **ensure software quality**, various **testing methods and quality assurance practices are employed** throughout the software development life cycle.

This includes **unit testing, integration testing, system testing, and acceptance testing**.

Additionally, best practices such as **code reviews, documentation, and adherence to coding standards** contribute to achieving and maintaining high software quality.

1. Unit Testing:

- **Isolation:** Tests individual components or modules in isolation.
- **Early Bug Detection:** Identifies and addresses defects at an early stage of development.

- **Automation:** Well-suited for automation, aiding in continuous integration and delivery.

2. Integration Testing:

- **Interface Verification:** Ensures that components work correctly when integrated.
- **Data Flow:** Validates the correct flow of data between integrated components.
- **Dependency Management:** Identifies and resolves issues related to component dependencies.

3. System Testing:

- **End-to-End Validation:** Validates the entire system's functionality in a real-world environment.
- **Performance Testing:** Evaluates system performance under various conditions.
- **Scalability:** Assesses the system's ability to scale with increasing load or data.

4. Acceptance Testing:

- **User Perspective:** Validates whether the software meets user requirements and expectations.

- **Business Alignment:** Ensures the software aligns with business goals and objectives.
- **Final Validation:** Represents the final phase of testing before the software is released.

Importance of software quality:

1. Customer Satisfaction:

High-quality software meets or exceeds customer expectations.

Satisfied customers are more likely to **continue using the software, recommend it to others**, and contribute positively to the reputation of the development team or organization.

2. Reduced Costs:

Identifying and fixing defects early in the development process is generally **less expensive** than **addressing them later** or after the software has been deployed.

Quality assurance practices can help catch issues early, preventing costly rework and post-release bug fixes.

3. Enhanced Reputation:

Software with a reputation for high quality is more likely to be trusted by users and stakeholders.

A **positive reputation can lead to increased user adoption**, better business partnerships, and a competitive advantage in the market.

4. Project Success:

The success of a software project is often measured not only by meeting functional requirements but also by delivering a reliable, efficient, and user-friendly product.

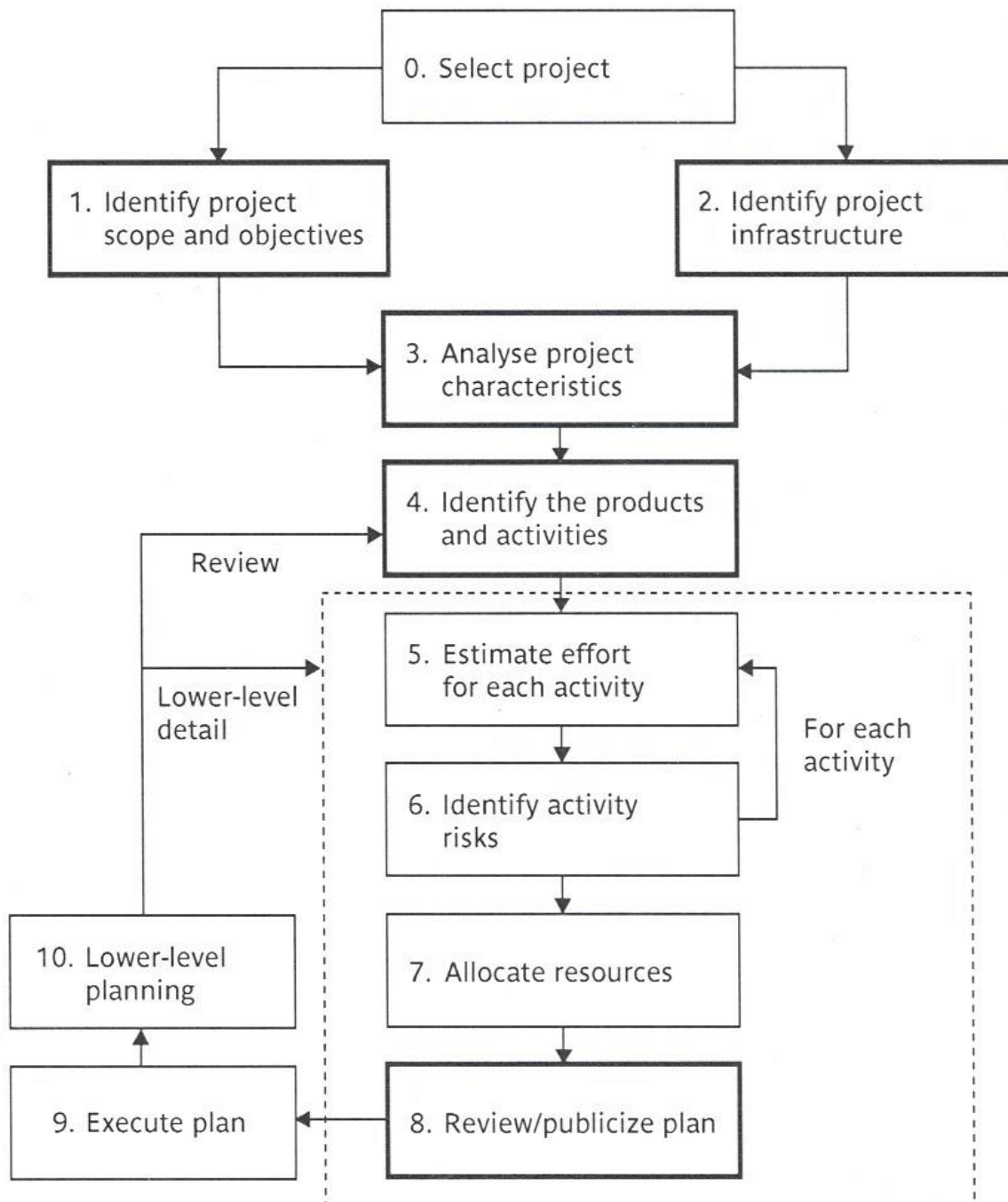
Software quality plays a crucial role in achieving project success by ensuring that the delivered software aligns with the project goals and objectives.

5. Maintainability and Extensibility:

High-quality software is typically easier to maintain and extend.

Well-organized code, clear documentation, and modular design make it more straightforward for developers to understand, update, and enhance the software over time.

The Place of Software Quality in Project/Software Planning “step wise framework”:



Quality will be of **concern in all stages** of the project **but** it will be of **particular interest at the following steps in bold** in the step wise framework.

- Step1
- Step2
- Step3
- Step4
- Step8

Step 1: Identify project scope and objectives: some objectives could relate to the qualities of the application to be delivered.

Step 2: Identify project infrastructure, within this step, activity 2.2 identifies installation standards and procedures. Some of those will be about quality.

Step 3: Analyze project characteristics, within this step activity 3.2 Analyze other project characteristics including quality-based ones.

Step 4: identify the products and activities of the project.

Step 8: Review and publicize plan, at this stage the overall quality aspects of the project plan are reviewed.

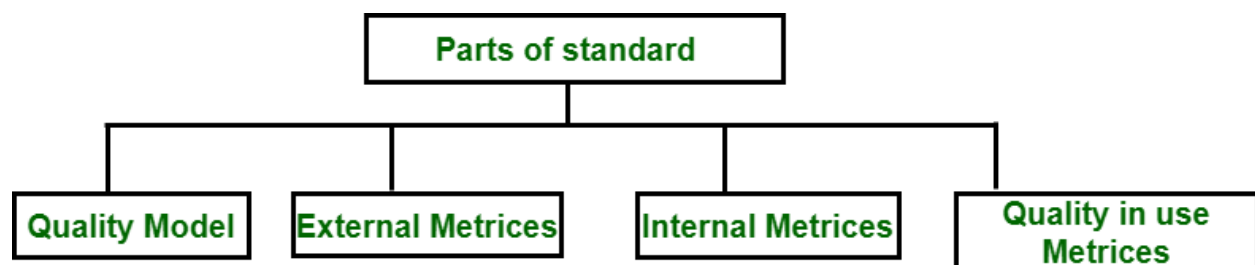
ISO 9126:

Stands for “International Organization for Standardization.”

There was a lack of agreed on definitions for the qualities of a good software.

This standard was introduced in 1991 to tackle the question of the definition of **software quality**.

The standard is divided into 4 parts as depicted in the following figure:



Part-1: Software Engineering: Product Quality

“Quality model”: It describes the quality model framework which explains relationships between different approaches to quality as well as identifying quality characteristics and sub-characteristics of software products.

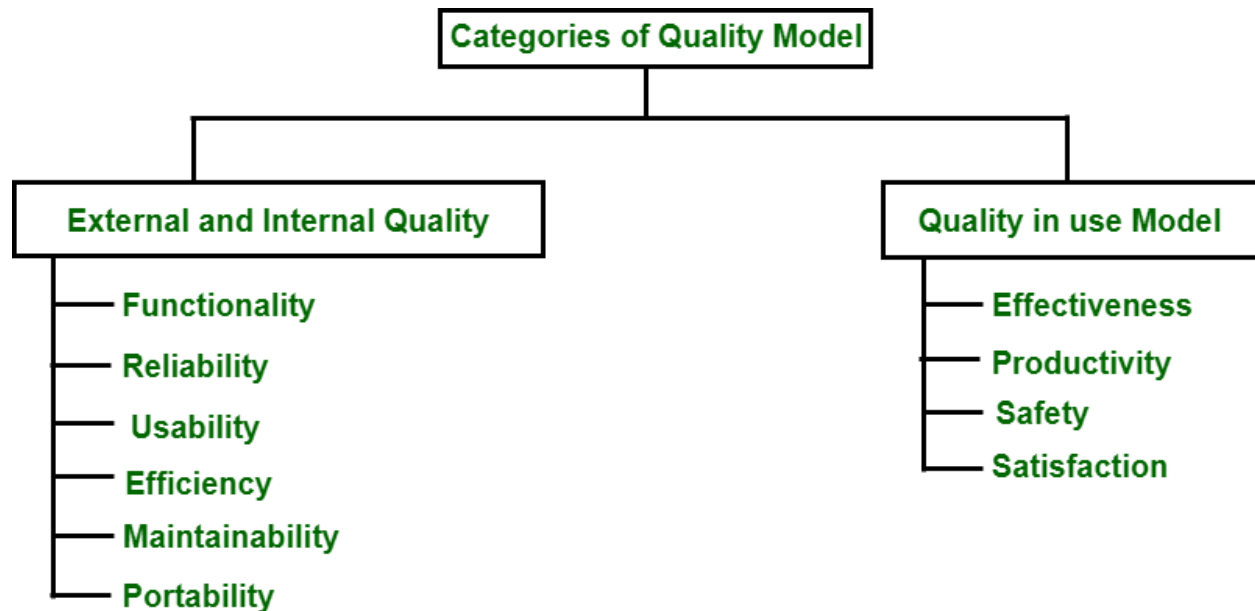
Part-2: Software Engineering: Product Quality
“External Metrics”: Its use is to describe external metrics that are used to measure characteristics and sub-characteristics which are identified in part 1.

Part-3: Software Engineering: Product Quality
“Internal Metrics”: Its use is to describe internal metrics that are used to measure characteristics and sub-characteristics which are identified in part 1.

Part-4: Software Engineering: Product Quality
“Quality in use metrics”: Its use is to identify metrics that are used to measure the effects of combined quality characteristics for users.

From the above discussion, it is concluded that the **first three parts are concerned with describing and measuring the quality of software products**, and the **fourth part is concerned with the quality of software products from the user’s point of view**.

Furthermore, the first part i.e. the Quality model is concerned classified into two categories as depicted in the following figure:



ISO 9126 defines six major software quality characteristics:

1. **Functionality:**

- **Definition:** Functionality refers to the set of features and capabilities that a software product provides to meet user needs and requirements.
- **Attributes:** Key attributes include suitability, accuracy, interoperability, compliance, and security.

- **Evaluation Criteria:** Assess the software's ability to perform specified functions accurately, efficiently, and in a manner that aligns with user expectations.

2. Reliability:

- **Definition:** Reliability is the ability of the software to maintain its level of performance under specified conditions for a defined period.
- **Attributes:** Reliability includes aspects like fault tolerance, recoverability, and adherence to specified performance standards.
- **Evaluation Criteria:** Evaluate the software's stability, resilience to failures, and its ability to recover from errors without significant impact on performance.

3. Usability:

- **Definition:** Usability is the effort required for users to learn, operate, and interact with the software effectively.
- **Attributes:** Usability characteristics include learnability, understandability, operability, and attractiveness.

- **Evaluation Criteria:** Assess how easily users can navigate the software, understand its functionalities, and perform tasks without undue cognitive load.

4. Efficiency:

- **Definition:** Efficiency relates to the use of physical resources by the software during its execution.
- **Attributes:** Efficiency includes aspects like time behavior, resource utilization, and response time.
- **Evaluation Criteria:** Evaluate how well the software performs its functions within specified resource constraints, such as processing speed, memory usage, and network bandwidth.

5. Maintainability:

- **Definition:** Maintainability is the effort required to make changes to the software, including corrections, enhancements, and adaptations.
- **Attributes:** Maintainability involves modifiability, stability, testability, and understandability.
- **Evaluation Criteria:** Assess the software's flexibility and ease of modification, ensuring that

changes can be made without causing unintended side effects.

6. Portability:

- **Definition:** Portability is the ability of the software to be transferred from one environment to another.
- **Attributes:** Portability includes adaptability, installability, coexistence, and replaceability.
- **Evaluation Criteria:** Evaluate the software's ability to run in different environments without requiring significant modifications, including different hardware or operating systems.